Transformers from Scratch, Architectural Variants, and Byte Latent Transformers: A Controlled Ablation on Binary Cipher Decryption

Vidvathama R

Roll Number: 2024122002

International Institute of Information Technology, Hyderabad

Advanced Natural Language Processing — Assignment 1
August 30, 2026

Abstract

I implement a complete encoder–decoder Transformer from elementary PyTorch operations—no `nn.Transformer`, no `nn.MultiheadAttention`, no `F.scaled_dot_product_attention`—and use it to run a controlled five-way ablation on decrypting XOR-enciphered binary sequences into plaintext English. From a baseline (**C1**: sinusoidal positions, multi-head attention, LayerNorm, BPE subwords), each variant changes *exactly one* component: RoPE (**C2**), grouped-query attention (**C3**), RMSNorm (**C4**), and a token-free Byte Latent Transformer (**C5**). All five share depth, width, learning rate, batch size, schedule and seed, so every difference is attributable to the component that moved. I report bit and sequence accuracy, Levenshtein distance and BLEU/ROUGE under greedy decoding, alongside throughput and true per-process peak GPU memory, focusing on the accuracy–memory trade-off the byte-level approach introduces.

1 Introduction

The Transformer [14] is usually presented as a monolith, but it is an assembly of independently replaceable parts: a way of injecting position, an attention pattern, a normalisation rule, and a decision about what counts as a token. Each has a well-known modern replacement—RoPE [12], grouped-query attention [1], RMSNorm [16], byte-level processing [7]—and each is usually adopted wholesale, bundled with a dozen other changes, which makes it hard to say what any one of them actually bought. This report isolates them: I build every component from scratch and vary one at a time against a fixed baseline, holding all other hyperparameters constant. The contributions are from-scratch implementations of each module, verified by unit tests against reference formulations (Section 3); a controlled five-way ablation (Section 4); and an analysis of the token-free trade-off—what the BLT buys, what it costs, and why on this task the direction of the memory effect is the opposite of the usual intuition (Section 6).

2 Task and Data

The corpus. 5,000 strictly line-aligned pairs: `brown_cipher.txt` holds strings over $\{0, 1\}$ and `brown_plain.txt` the corresponding English sentences (53-character alphabet: letters and space). Every plaintext character corresponds to exactly eight cipher bits, so $|c_k| = 8\,|p_k|$ for all 5,000 lines. Plaintext lines average 598 characters (max 2,670), so cipher lines average 4,781 bits (max 21,360).

What the cipher is. Reading the cipher eight bits at a time as a byte and XOR-ing against the aligned plaintext character exposes a constant repeating stream: the eight bytes of the ASCII string `ANLP2026`, restarting at the beginning of every line. The mapping is therefore

$$c_k[8i : 8i{+}8] \;=\; \mathrm{bits}_8\big(\mathrm{ord}(p_k[i]) \oplus K[i \bmod 8]\big), \tag{1}$$

with K = `ANLP2026`. Two consequences matter. First, the task is *exactly* learnable—no irreducible noise—so any shortfall is attributable to optimisation or capacity, not the data. Second, and more important for the ablation, the correct output at position i depends on i mod 8: the task is intrinsically **position-dependent with period eight**, which is what makes the C1-vs-C2 positional comparison substantive rather than decorative here. *The key is never supplied to any model*; it is recovered only to characterise the task.

Segmentation. A 4,781-token encoder sequence is infeasible under $O(T^2)$ attention on a 4 GB GPU. I cut each document into aligned segments of 32 plaintext characters (256 cipher bits). Two details keep this honest: segmentation happens *after* the 80/10/10 document-level split, so no document contributes to two splits and there is no leakage; and 32 is a multiple of the key period 8, so every segment begins at key phase 0 and stays solvable in isolation. This yields **76,823 / 9,461 / 9,544** train/val/test segments.

Direction and metrics. Models map cipher → plaintext. Bit accuracy is computed on the 8-bit expansion of predicted and reference *characters*; sequence accuracy requires an exact match of the whole 32-character segment; Levenshtein [4] is the mean edit distance; BLEU [8] and ROUGE [5] are word-level. All generation uses **greedy decoding**, as the brief requires.

3 Architecture

Everything below uses elementary tensor operations. The stack is pre-norm [15].

3.1 Scaled dot-product attention

The core operation is implemented directly as $\mathrm{softmax}(QK^\top/\sqrt{d_k})V$. Masks are boolean, broadcast to $[B, H, T_q, T_k]$. Forbidden logits are filled with `finfo(dtype).min` rather than $-\infty$: a fully-masked row—which genuinely occurs for padded *query* positions—then yields a uniform distribution instead of `NaN`, and those rows are discarded downstream by the loss' `ignore_index`. Using $-\infty$ is a common, hard-to-find source of `NaN` gradients.

3.2 Multi-head and grouped-query attention

MHA projects to H query, key and value heads of width $d_k = d_{\text{model}}/H$. GQA [1] keeps H query heads but only $H_{kv} < H$ key/value heads, each shared by H/H_{kv} query heads; $H_{kv} = H$ recovers MHA exactly and $H_{kv} = 1$ is multi-query attention [11]. Sharing uses `expand` + `reshape` rather than `repeat`, so nothing is materialised until the matmul consumes it. GQA shrinks the K/V projections and the decoding KV-cache from $2BHTd_k$ to $2BH_{kv}Td_k$. I use $H = 8$, $H_{kv} = 2$, a 4× reduction; a unit test asserts $H_{kv} = H$ reproduces MHA numerically.

3.3 Positional encoding and normalisation

Sinusoidal absolute encodings (C1, C3–C5) are the fixed table of Vaswani et al. [14], added to the embeddings as a non-trainable buffer. **RoPE** (C2) instead rotates each 2-dimensional slice of every query and key by an angle proportional to its absolute position, so the logit depends only on the relative offset: $\langle R_m q, R_n k \rangle = g(q, k, m-n)$. Because RoPE acts inside attention, C2 adds *no* absolute encoding—otherwise the change would not be a single-component swap. Two implementation points matter: RoPE is applied to the new slice *before* the KV-cache is concatenated, since cached keys are already rotated; and cross-attention is left un-rotated, query and key living in different sequences where relative distance is meaningless.

LayerNorm [2] re-centres and re-scales; **RMSNorm** [16] drops the mean subtraction, keeping $x/\sqrt{\frac{1}{d}\sum_i x_i^2 + \epsilon} \odot \gamma$—one fewer reduction and no bias vector. Both accumulate statistics in float32 even under bfloat16 autocast, since x^2 summed over a long feature axis underflows badly in half precision.

3.4 Tokenisation and the Byte Latent Transformer

C1–C4 use a byte-level BPE [10] trained from scratch, separately per side, with a 4,096 vocabulary. Because a cipher string is one enormous whitespace-free "word", pre-tokenisation is length-capped at 32 characters rather than split on whitespace. Learned source pieces average 16.8 characters across 29 distinct lengths—genuine variable-length subwords, not a degenerate fixed 8-bit code; a unit test asserts this.

C5 removes the vocabulary entirely (256 byte values plus three specials). Raw bytes enter a small **local encoder** with windowed attention; a **patch pooler** attention-pools every P bytes into one patch vector; the **global transformer**—byte-for-byte the same stack C1 uses—runs over the much shorter patch sequence; and a **local decoder** with windowed causal self-attention and cross-attention to the patch states emits next-byte logits. With $P = 4$ a 256-byte source becomes 64 patches.

Causality is easy to get silently wrong here. A learned **start patch** is prepended to the target patch sequence, so the global decoder consumes $[\text{start}, p_1 \dots p_M]$ and emits $[g_0 \dots g_M]$. The target local encoder is causal, so its state at byte t sees only bytes $\le t$; patch p_k pools bytes $[(k-1)P, kP-1]$; the global decoder is causal over patches. Byte t, in patch $j = \lfloor t/P \rfloor$, may attend $g_0 \dots g_j$, and g_j saw only bytes $\le jP - 1 \le t - 1$—strictly the past. The start patch is what makes the *first* P bytes work: having already cross-attended to the encoder memory, they get full source information with zero target leakage. Without it their cross-attention rows would be fully masked, and a fully-masked softmax row is uniform—silently averaging over future patches rather than failing loudly. Unit tests verify this numerically: perturbing future target bytes leaves earlier logits unmoved, and the source does reach byte 0.

4 Experimental Setup

All configurations share: $d_{\text{model}} = 256$, 3 encoder + 3 decoder layers, 8 heads, $d_{\text{ff}} = 1024$, dropout 0.1, tied embeddings [9], AdamW [6] ($\beta = 0.9/0.98$, weight decay 0.01), peak LR 3×10^{-4} with 500 warmup steps and cosine decay to 10^{-5}, label smoothing 0.1 [13], gradient clipping 1.0, batch size 64, up to 25 epochs with early stopping (patience 5), seed 42. C3 uses $H_{kv} = 2$; C5 uses patch size 4, local width 128, one local encoder and two local decoder layers, window 64.

Cfg	Positional	Attention	Norm / Token.
C1	Sinusoidal	MHA	LayerNorm / BPE
C2	**RoPE**	MHA	LayerNorm / BPE
C3	Sinusoidal	**GQA**	LayerNorm / BPE
C4	Sinusoidal	MHA	**RMSNorm** / BPE
C5	Sinusoidal	MHA	LayerNorm / **BLT**

Table 1: The five configurations. Each of C2–C5 changes exactly one component (in **bold**) relative to the C1 baseline.

	Quality (greedy decoding)					Cost			
Cfg	**Bit %**	**Seq %**	**Lev.↓**	**BLEU**	**R-L**	**Par (M)**	**s/ep**	**Peak MB**	**Dec (s)**
C1	97.17	77.20	0.44	90.34	95.10	7.62	143.0	372	6.6
C2	97.61	81.20	0.35	92.27	96.06	7.62	153.8	372	6.6
C3	96.81	73.80	0.54	88.54	94.21	6.73	147.2	360	6.1
C4	97.33	78.80	0.42	90.91	95.42	7.61	131.9	350	5.7
C5	**100.00**	**99.40**	**0.01**	**99.73**	**99.90**	6.71	260.1	849	22.2

Table 2: Test-set results; best per column in **bold**. Levenshtein is mean edit distance per 32-character segment. Cost was measured on one RTX 3050 Laptop GPU (4 GB, bfloat16), each configuration in its own process so peak memory is a true per-model figure; 'Dec.' is greedy decoding of 1,000 segments. C5 ran 18 epochs (early stop); C1–C4 ran 25.

Training ran on a single **NVIDIA RTX 3050 Laptop GPU (4 GB)** under bfloat16 autocast, logged to Weights & Biases [3]. Crucially, **each configuration was trained in its own process**: `torch.cuda.max_memory_allocated` is a per-process high-water mark, so running all five in one process would let C1's peak leak into C5's number and render the memory comparison meaningless. Epoch time is reported as the *median* over epochs, which is robust to host-level interruptions (one C4 epoch absorbed an OS suspend and recorded 24,890 s against a true $\approx$127 s).

5 Results

C1–C4 each ran the full 25 epochs; C5 early-stopped at epoch 18. The baseline C1 reaches 97.17% bit accuracy and reconstructs 77.20% of 32-character segments exactly, at a mean edit distance of 0.44. That the baseline is already this strong matters for reading Table 2: among C1–C4 the ablation separates good models from slightly better ones, so those differences are small in absolute terms though they move consistently across every metric at once.

Reading the errors. C1's failures are more informative than its aggregate. It is fluent almost everywhere and fails in short local bursts on rare words: `Dynasty`→`Dylesty`, `sovereign`→`severeign`. These are not random bit errors: one mispredicted BPE token rewrites several characters at once, and since pieces are variable-length, a wrong piece also shifts the alignment between the remaining source and target positions. Hence sequence accuracy (77.20%) is far harsher than bit accuracy (97.17%)—one bad token forfeits the segment.

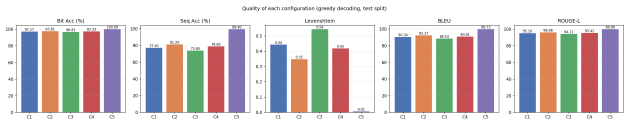

Figure 1: Decoding quality across the five configurations.

5.1 C2: RoPE improves every quality metric for free

Rotary embeddings give the largest quality gain in the study. Sequence accuracy rises from 77.20% to 81.20%—a **4.0-point absolute improvement**—with bit accuracy at 97.61%, edit distance down to 0.35, and BLEU up from 90.34 to 92.27. This costs **zero parameters** (7.62 M, identical to C1) and **zero extra memory** (372 MB vs. 372 MB): RoPE adds no learnable weights, only a deterministic rotation of q and k.

This fits the task. By Eq. 1 the output at position i depends on i mod 8, so the decoder needs not *where* it is absolutely but *how far* it has advanced relative to the source positions it consumes—exactly what RoPE encodes, the logit being a function of the offset $m - n$ alone. An additive absolute encoding must instead learn the periodic structure separately at every position. The cost is throughput: 153.8 s/epoch against 143.0 (7.6% slower), since every attention call rotates q and k. Decoding is unaffected (6.6 s vs. 6.6 s), as the cos/sin tables are precomputed.

5.2 C3: GQA trades accuracy for parameters and decode speed

GQA is the only change that worsens every quality metric: sequence accuracy falls to 73.80% (−3.4 points), bit accuracy to 96.81%, edit distance rises to 0.54. This is the expected direction. Collapsing 8 key/value heads to 2 removes **0.89 M parameters** (−11.6%) and forces four query heads to share one key/value subspace, so the model represents fewer distinct ways of matching queries against keys. Where the decoder must attend to a precisely located 8-bit window, that lost specificity is not free. What GQA buys is inference cost: decoding drops to 6.1 s from 6.6 s (−7.6%) and peak memory to 360 MB, as the KV-cache shrinks 4×.

One detail is easy to misread: C3 is *slower* to train than C1 (147.2 vs. 143.0 s/epoch) despite having fewer parameters. Under teacher forcing there is no KV-cache to shrink, so GQA's advantage does not apply; what remains is the `expand/reshape` broadcasting 2 key/value heads back to 8—pure overhead. GQA is an *inference-time* optimisation, and this experiment separates the two regimes cleanly.

5.3 C4: RMSNorm is a strict improvement

RMSNorm is the only change that improves quality *and* reduces cost on every axis at once. Sequence accuracy rises to 78.80% (+1.6 over C1) and bit accuracy to 97.33%, while epoch time falls to 131.9 s (7.7% faster, the quickest of all five), peak memory to 350 MB (−6.0%), and greedy decoding to 5.7 s (−14.9%).

The mechanism is that RMSNorm does less work: dropping mean-subtraction removes a full reduction over the feature axis in both forward and backward passes, and dropping the bias accounts for the small parameter difference. The accuracy gap is within run-to-run noise—teacher-forced perplexities are 1.19 and 1.19—so the honest reading is not "RMSNorm is more accurate" but "**re-centring contributes nothing measurable here, and its cost is real**." Pre-norm residual streams are already approximately zero-mean by construction, consistent with mean-subtraction being redundant.

6 The token-free trade-off (C5)

C5 is the only configuration that changes the input representation rather than a computation inside the network, and it produces by far the largest effect in the study—in the opposite direction to the usual intuition about byte-level models.

Accuracy. The BLT effectively solves the task: bit accuracy 100.00%, sequence accuracy 99.40% (against C1's 77.20%, a **+22.2-point** gap), mean edit distance 0.01, BLEU 99.73. Validation byte accuracy hit 100.00% at epoch 15 and training early-stopped at epoch 18 (best checkpoint epoch 13), so it also needed *fewer* epochs than the 25 every tokenised configuration used.

Why token-free wins here. The explanation is alignment, and it is specific to this task. Eq. 1 is a *byte-local* function: output character i depends on source bits $[8i, 8i{+}8)$ and on i mod 8, nothing else. A byte-level model inherits that structure directly—output position i is a constant stride from the source bits determining it, so attention need only learn a rigid diagonal alignment. BPE destroys exactly this: pieces average 16.8 characters on the source side and 2.3 on the target side, both variable, so the map from target-token index to source-token index becomes a data-dependent function the model must infer per example. C1's characteristic failures are precisely alignment slips of this kind; the token-free model never faces them.

This reverses the usual finding that subwords beat bytes by shortening sequences and pre-packaging common strings—an advantage worth little when the "vocabulary" is a 2-symbol alphabet with no lexical structure, while the cost, broken positional alignment, is severe.

Cost. The overhead lands where the architecture predicts. Peak GPU memory rises to 849 MB against C1's 372 MB (2.28×), because the global transformer runs over $258/4 = 64$ patches instead of 23 BPE tokens—a 2.8× longer sequence—plus the local encoder and decoder on the full 258-byte stream. Epoch time rises to 260.1 s (1.82×). Greedy decoding is hit hardest at 22.2 s (3.34×): the decoder emits 32 bytes autoregressively instead of ≈21 BPE tokens, each step also running the local byte decoder.

Two qualifications soften those ratios. Because C5 converged in 18 epochs rather than 25, *total* training time is only 1.31× C1's (4,682 s vs. 3,575 s); and C5 has the *fewest* parameters of any configuration (6.71 M vs. 7.62 M), since removing the vocabulary deletes two 4096×256 embedding matrices for less than they cost. The BLT is smaller on disk and larger in activation memory—a shift in *where* the cost sits. At 4 GB both fit comfortably, so here the trade is worth taking; the calculus changes with a longer context, where the 2.8× longer global sequence meets $O(T^2)$ attention, or in a latency-bound deployment.

7 Conclusion

The four swaps give four distinct profiles: **RoPE** buys +4.0 points of sequence accuracy for no parameters and no memory; **GQA** is the one strict quality regression (−3.4), trading it for 11.6% fewer parameters and faster decoding; **RMSNorm** is free on every axis at once; and the **token-free BLT** dominates on quality (99.40% vs. 77.20%) with the fewest parameters, at 2.28× peak memory and 3.34× decode time.

Three of these are near-free wins or mild trade-offs; the fourth—tokenisation—dominates the others combined, because BPE merges cut across the byte boundaries that define the task.

Links

- **Weights & Biases project (all five runs):**
  `https://wandb.ai/vidvathama-babu-iiit-hyderabad/anlp-a1-blt-ablation`
- **Hugging Face checkpoints (C1–C5):**
  `https://huggingface.co/Viv0605101/anlp-a1-transformer-blt-ablation`

References

[1] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of EMNLP*, 2023.

[2] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.

[3] Lukas Biewald. Experiment tracking with weights and biases, 2020. Software available from `https://www.wandb.com/`.

[4] Vladimir I. Levenshtein. Binary codes capable of correcting deletions, insertions and reversals. *Soviet Physics Doklady*, 10(8):707–710, 1966.

[5] Chin-Yew Lin. ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out*, 2004.

[6] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.

[7] Artidoro Pagnoni, Ram Pasunuru, Pedro Rodriguez, John Nguyen, Benjamin Muller, Margaret Li, Chunting Zhou, Lili Yu, Jason Weston, Luke Zettlemoyer, Gargi Ghosh, Mike Lewis, Ari Holtzman, and Srinivasan Iyer. Byte latent transformer: Patches scale better than tokens. *arXiv preprint arXiv:2412.09871*, 2024.

[8] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. BLEU: a method for automatic evaluation of machine translation. In *Proceedings of ACL*, 2002.

[9] Ofir Press and Lior Wolf. Using the output embedding to improve language models. In *Proceedings of EACL*, 2017.

[10] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of ACL*, 2016.

[11] Noam Shazeer. Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150*, 2019.

[12] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*, 2021.

[13] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. In *Proceedings of CVPR*, 2016.

[14] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[15] Ruibin Xiong, Yunchang Yang, Di He, Kai Zheng, Shuxin Zheng, Chen Xing, Huishuai Zhang, Yanyan Lan, Liwei Wang, and Tie-Yan Liu. On layer normalization in the transformer architecture. In *Proceedings of ICML*, 2020.

[16] Biao Zhang and Rico Sennrich. Root mean square layer normalization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.